

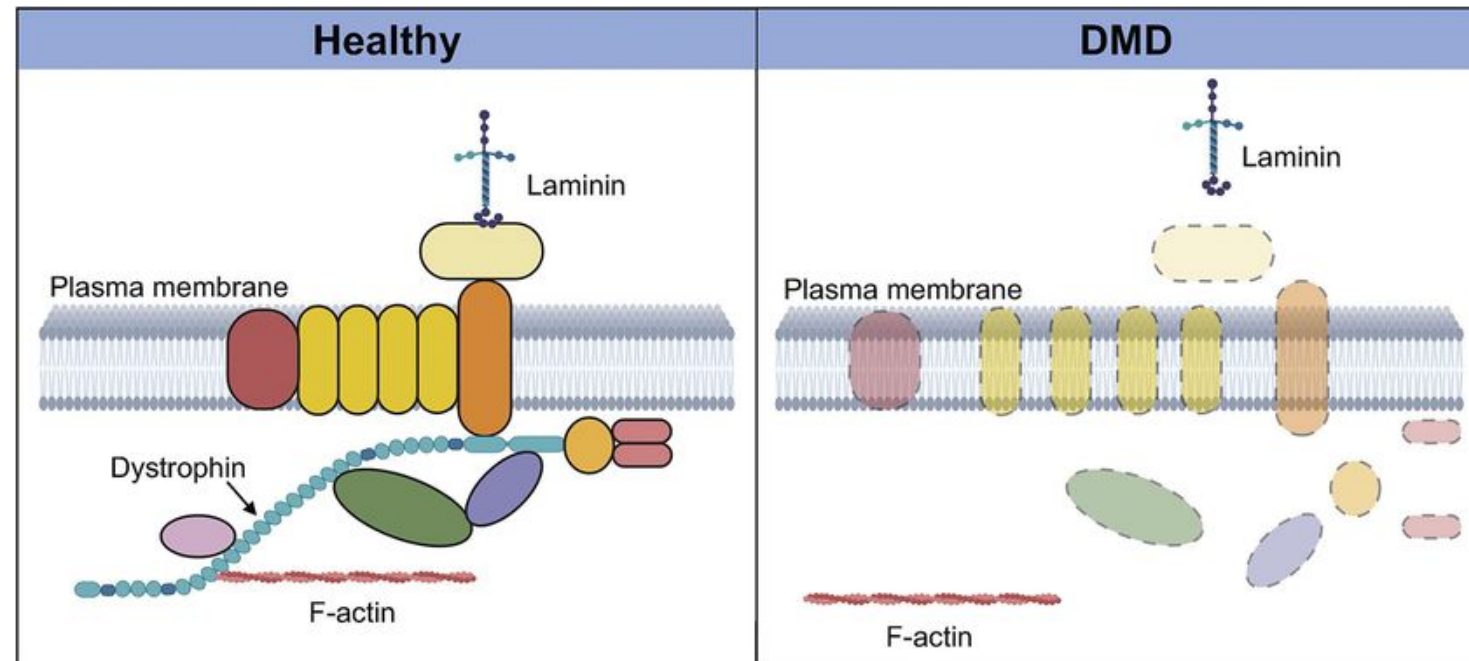
MyoMetricsMR

An AI-powered MRI analysis software pipeline to help pediatric neurologists, radiologists and clinical researchers detect and monitor Duchenne Muscular Dystrophy early, addressing the limitations of delayed, costly, and invasive diagnostic methods.





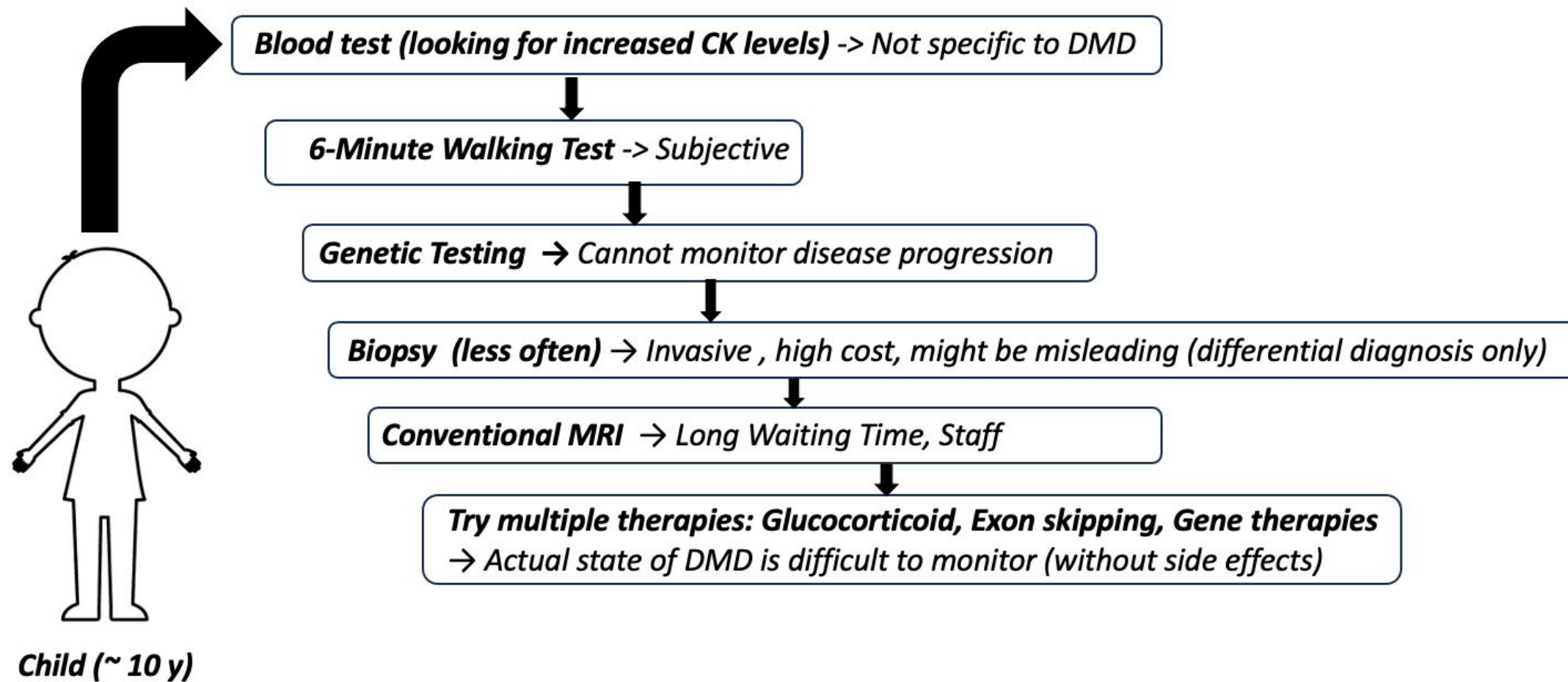
What is Duchenne Muscular Dystrophy?



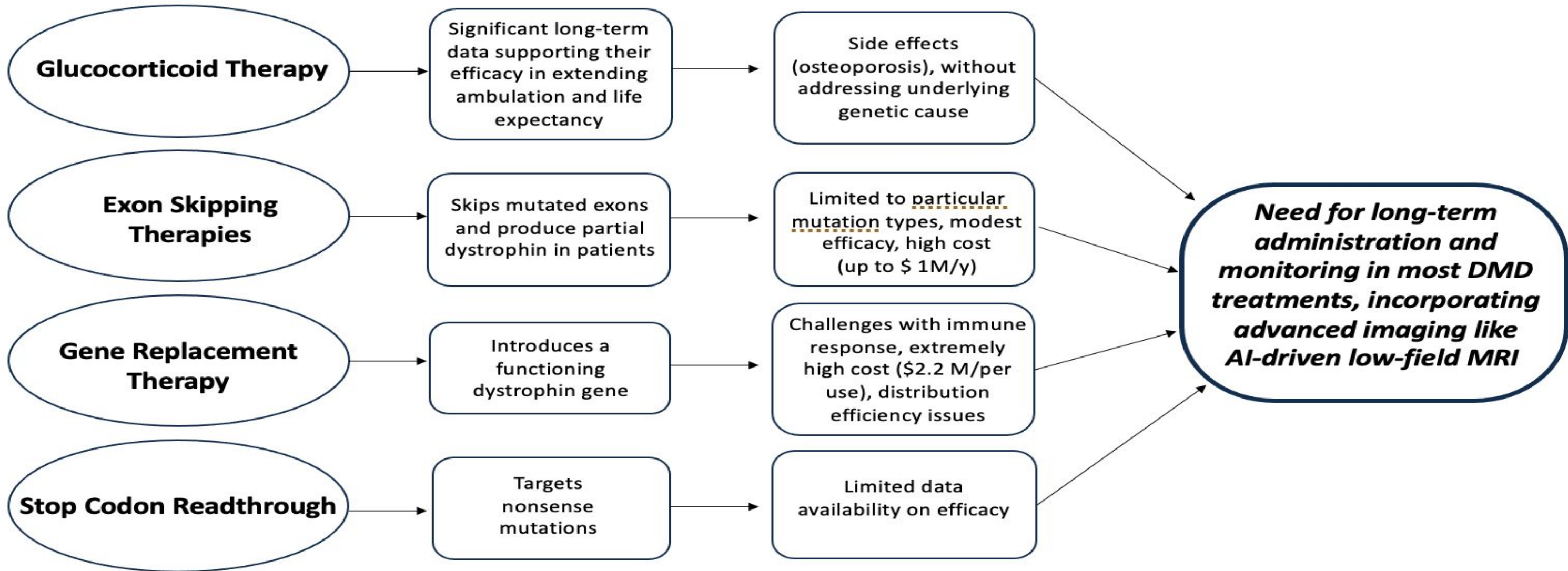
Source: Kodippili and Rudnicki

Problems with current diagnostics and how we fix them

Current Approaches to DMD Diagnostics - Workflow



Current Therapeutic Approaches for DMD



Problems with current diagnostics and how we fix them

Current approaches to DMD diagnostics

*6-Minute Walking Test
→ Low Accuracy*

*Biopsy → Invasive and
High Cost*

*Biopsy → might be
misleading (differential
diagnosis only)*

*Genetic Testing →
Cannot monitor
disease progression*

*Conventional MRI →
Long Waiting Time,
Staff*



MyoMetrics MRI Monitoring

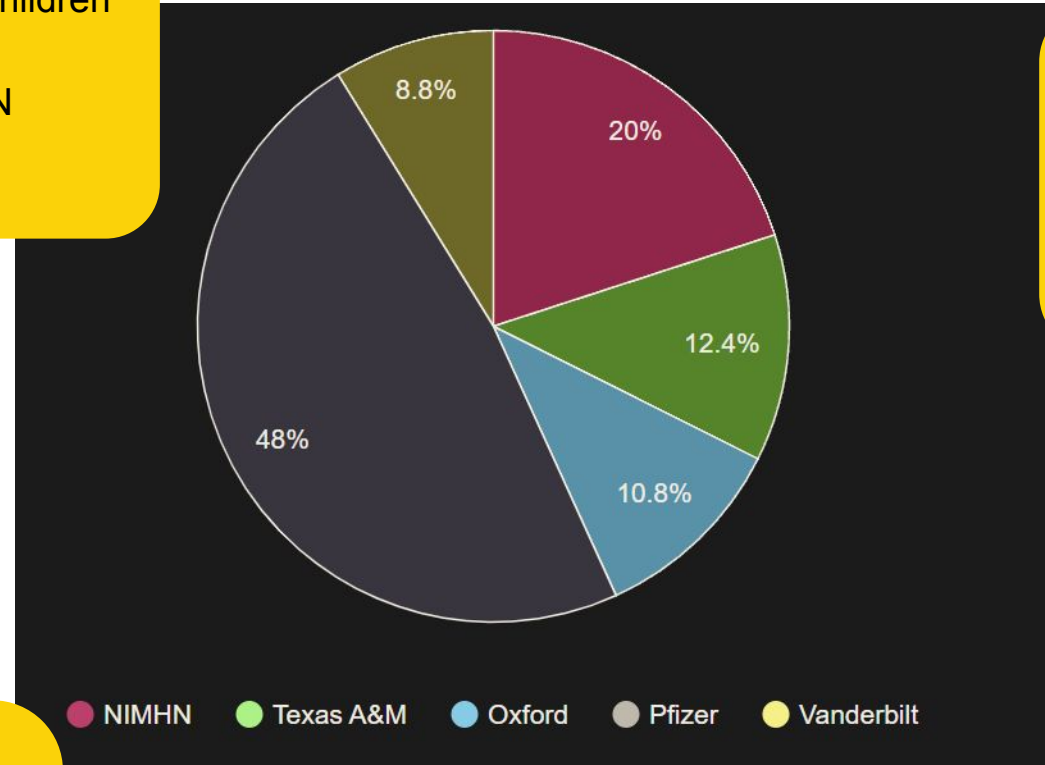
- Highly cost-effective
- Detailed tissue analysis
- Fastened by AI
- High availability
- Efficient processing
- Non-invasive
- Allows for disease tracking
- Alleviates burden on radiologists
- Higher diagnostic capacity in remote areas
- Supports early intervention

Training Data Sources

- 50 genetically confirmed DMD children
- 1.5 T MRI machine (SIEMENS)
- T2-weighted images with DIXON Sequences

- Avg 31.1 subjects per study
- 1.5 T or 3.0 T MRI Scanners
- T2-weighted images with DIXON Sequences

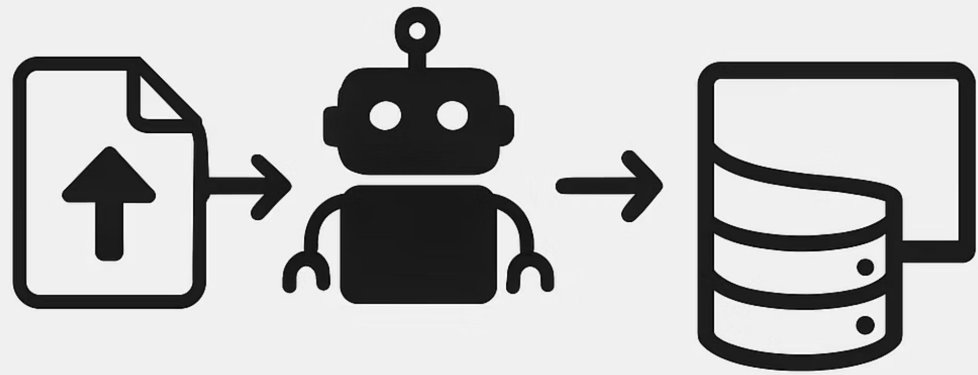
- 120 entries (largest dataset available)
- 1.5 - 3.0 T MRI Scans
- T2-weighted images with DIXON Sequences



- 27 participants
- 3.0 T MRI Scans
- T2-weighted images with DIXON Sequences

- T1 and T2 MRI from Vanderbilt Tennessee Paper.
- Number of participants equal to 22. Strong correlations despite small dataset size.

Source: Jurasova et al.



Secure Medical Imaging Upload & Analysis Pipeline

Built with FastAPI, PostgreSQL, and Machine Learning

Terezia Juras

Addressing Critical Challenges



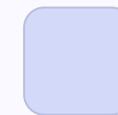
Secure Data Handling

Processing sensitive medical images like MRIs demands strict adherence to security and compliance standards, including HIPAA and robust encryption protocols.



Automated Processing

The pipeline must support automated DICOM conversion, patient information anonymization, and seamless integration with machine learning diagnostics.



Cost-Effective Solutions

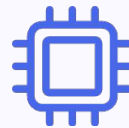
To meet budget constraints, the architecture leverages open-source tools and containerized microservices for efficient and scalable deployment.

Core System Components



Frontend Interface

Wix serves as the intuitive patient upload interface, ensuring a user-friendly experience.



Processing Microservice

FastAPI powers the backend, managing secure uploads and all image processing workflows.



Data Management

Encrypted files are stored securely, with PostgreSQL managing metadata, logs, and analysis results.



ML Integration

A placeholder machine learning model provides mock diagnoses, demonstrating future capabilities.

Seamless Medical Imaging

Workflow

1

Upload

Patients securely upload images via the Wix frontend.

2

Processing

FastAPI handles the file, converts to DICOM, and anonymizes patient data.

3

Encryption & Storage

Files are encrypted, with metadata and logs securely saved to PostgreSQL.

4

ML Analysis

The system simulates an ML diagnosis and returns results to the user in 3 days. (not implemented yet)

This streamlined process ensures efficient, secure, and rapid image analysis from upload to diagnosis.

Upload

Patients securely upload images via the Wix frontend only if logged in.

Secure Medical Image Upload

Choose Image +

Please log in to upload MRI images.

Diagnosis result

Confidence result

Upload ID

Submit

Secure Medical Image Upload

Choose Image +

Permission denied: Only verified and active clinicians can upload MRI images.

Diagnosis result

Confidence result

Upload ID

Submit

```
try {
  // Get the current user's email
  const email = await currentUser.getEmail();

  // Check clinician collection for permissions
  const clinicianResults = await wixData.query("Clinicians")
    .eq("email", email)
    .eq("isVerified", true)
    .eq("disabled", false)
    .find();

  if (clinicianResults.items.length === 0) {
    $w("#promptClinicalTests").text = "Permission denied: Only verified and active clinicians can upload MRI images.";
    $w("#promptClinicalTests").show();
    return;
  }

  $w("#submitButton").enable();
  $w("#uploadButton1").enable();
} catch (error) {
  console.error("Error checking clinician permissions:", error);
  $w("#promptClinicalTests").text = "Error verifying clinician access.";
  $w("#promptClinicalTests").show();
  return;
}
```

Sign in with Google

Select an account

to access the application [User Login](#)

Terézia Jurášová
terezka-jurasova@gmail.com

MEDCON
mmedcon@gmail.com

Terezia Juras
tj2286@nyu.edu

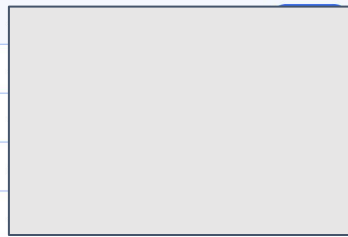
Use another account

Before using the User Login app, you may wish to review its [Privacy Policy](#) and [Terms of Use](#).

Clinicians

Default view 7 items + New View

Default view Table Refresh order Manage Fields

☐	User Id	Email	☒ is verified	☒ Disabled
1		mmedcon@gmail.com	☒	
2		tj2286@nyu.edu		
3		tj2286@nyu.edu		
4		tj2286@nyu.edu	☒	
5		tj2286@nyu.edu		
6		mmedcon.it@gmail.com	☒	

Secure Medical Image Upload

Choose Image +

Diagnosis result

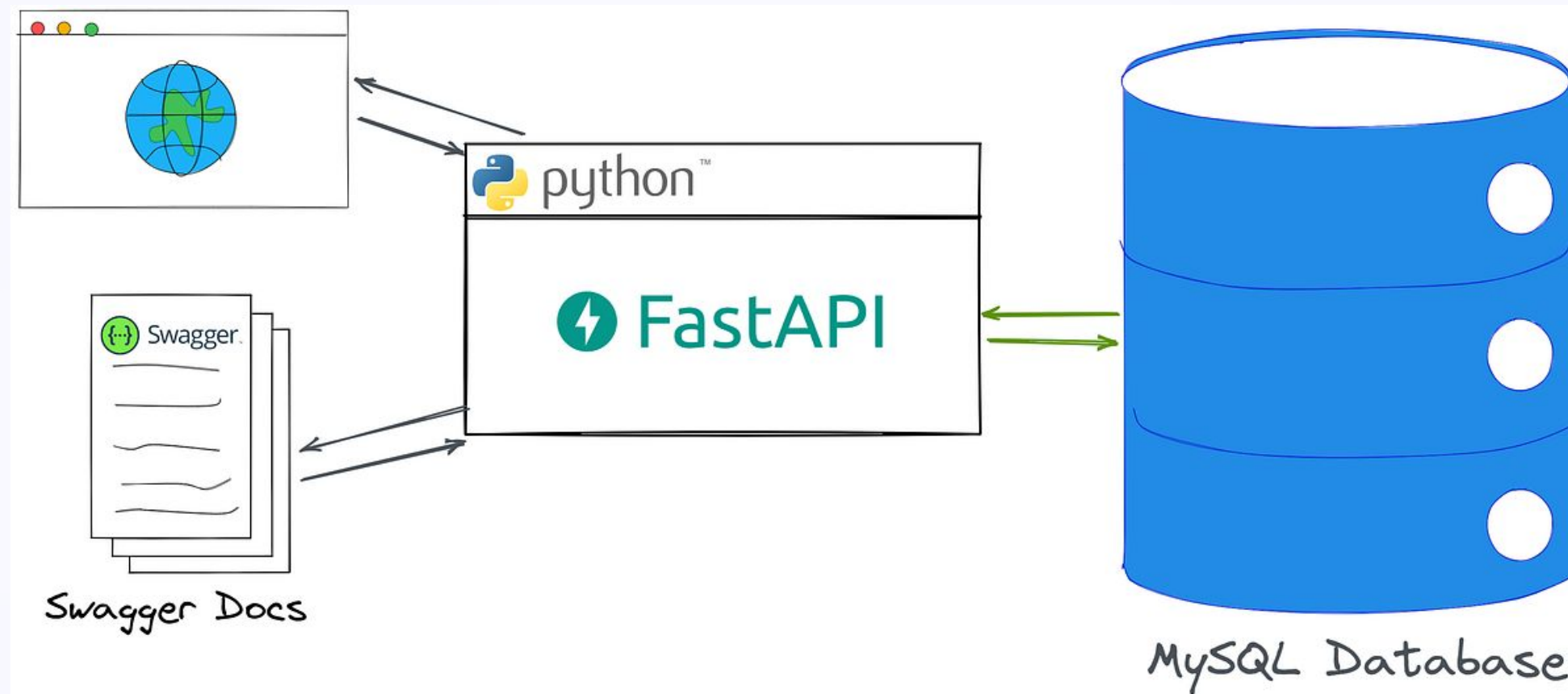
Confidence result

Upload ID

Submit

Processing

FastAPI handles the file, converts to DICOM, and anonymizes patient data.



Encryption & Storage

Files are encrypted, with metadata and logs securely saved to PostgreSQL.

WIX

Dicom Metadata

More Actions

Default view

5 items

+ New View

Default view

Table

Refresh order

Manage Fields

Sort

Filter

Mirror on Site

☐	Upload ID	☒ DICOM Converted	☒ Anonymized	Removed Tags	Conversion Log
1	file_001.dcm	✓	✓	PatientName,PatientID,Ac...	Conversion successful wit...
2	scan_20240601_002.dcm				File was not a valid DICO...
3	mri_heart_003.dcm	✓	✓	PatientName,PatientBirth...	Tags removed and file con...
4	ct_brain_004.dcm	✓			Conversion done but ano...
☐	ultrasound_005.dcm	✓	✓	PatientName,ReferringPh...	All sensitive tags stripped ...
+ Add Item					

Aiven hosts PostgreSQL

pg-mmedcon PostgreSQL 16.9 EOL : 9 November 2028 : OK Running Nodes 1

My Organization / finance25 / pg-mmedcon / Logs

Enable logs integration

Logs

[13-1] pid=50041,user=postgres,db=defaultdb,app=system-stats,client=[local] LOG: disconnection: session time: 0:00:00.065 user=postgres database=defaultdb host=[local]

[pg-mmedcon-5] 2025-07-20T18:14:51Z [postgresql-16]

[11-1] pid=50046,user=[unknown],db=[unknown],app=[unknown],client=[local] LOG: connection received: host=[local]

[pg-mmedcon-5] 2025-07-20T18:14:51Z [postgresql-16]

[12-1] pid=50046,user=postgres,db=finance,app=[unknown],client=[local] LOG: connection authorized: user=postgres database=finance

[pg-mmedcon-5] 2025-07-20T18:14:51Z [postgresql-16]

[13-1] pid=50046,user=postgres,db=finance,app=system-stats,client=[local] LOG: disconnection: session time: 0:00:00.011 user=postgres database=finance host=[local]

[pg-mmedcon-5] 2025-07-20T18:14:51Z [postgresql-16]

[11-1] pid=50047,user=[unknown],db=[unknown],app=[unknown],client=[local] LOG: connection received: host=[local]

[pg-mmedcon-5] 2025-07-20T18:14:51Z [postgresql-16]

[12-1] pid=50047,user=postgres,db=mmedconv2,app=[unknown],client=[local] LOG: connection authorized: user=postgres database=mmedconv2

[pg-mmedcon-5] 2025-07-20T18:14:51Z [postgresql-16]

[13-1] pid=50047,user=postgres,db=mmedconv2,app=system-stats,client=[local] LOG: disconnection: session time: 0:00:00.012 user=postgres database=mmedconv2 host=[local]

[pg-mmedcon-5] 2025-07-20T18:14:51Z [postgresql-16]

[11-1] pid=50048,user=[unknown],db=[unknown],app=[unknown],client=[local] LOG: connection received: host=[local]

[pg-mmedcon-5] 2025-07-20T18:14:51Z [postgresql-16]

[12-1] pid=50048,user=postgres,db=mmedcon,app=[unknown],client=[local] LOG: connection authorized: user=postgres database=mmedcon

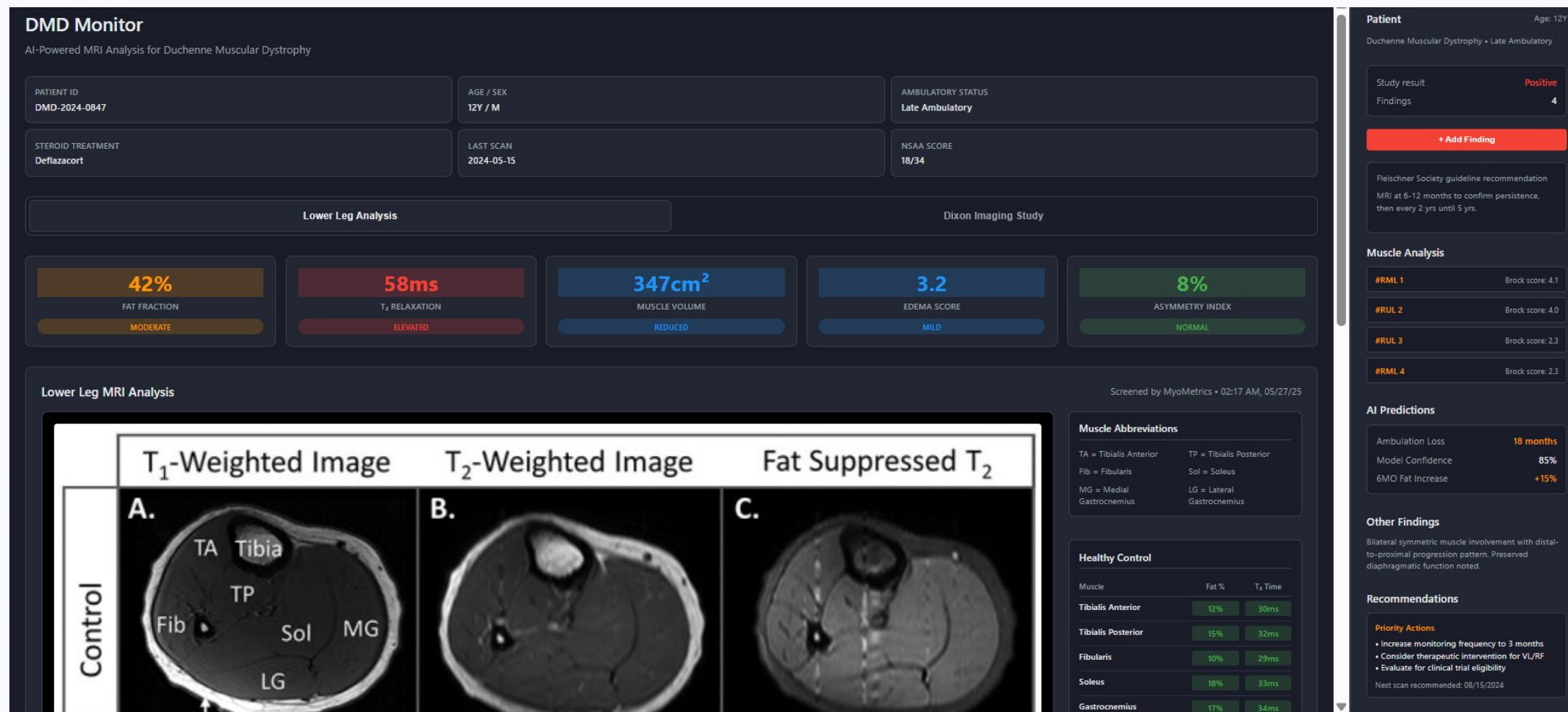
[pg-mmedcon-5] 2025-07-20T18:14:51Z [postgresql-16]

[13-1] pid=50048,user=postgres,db=mmedcon,app=system-stats,client=[local] LOG: disconnection: session time: 0:00:00.010 user=postgres database=mmedcon host=[local]

Go to most recent message

ML Analysis

The system simulates an ML diagnosis and returns results to the user in 3 days.
(not implemented yet)



Security Measures

TLS 1.2+ Encryption: Ensures secure data transmission over networks, protecting sensitive medical information in transit.



The [PCI Council](#) states that any organization handling payment card transactions must use TLS 1.2 or higher in order to meet PCI DSS compliance standards.

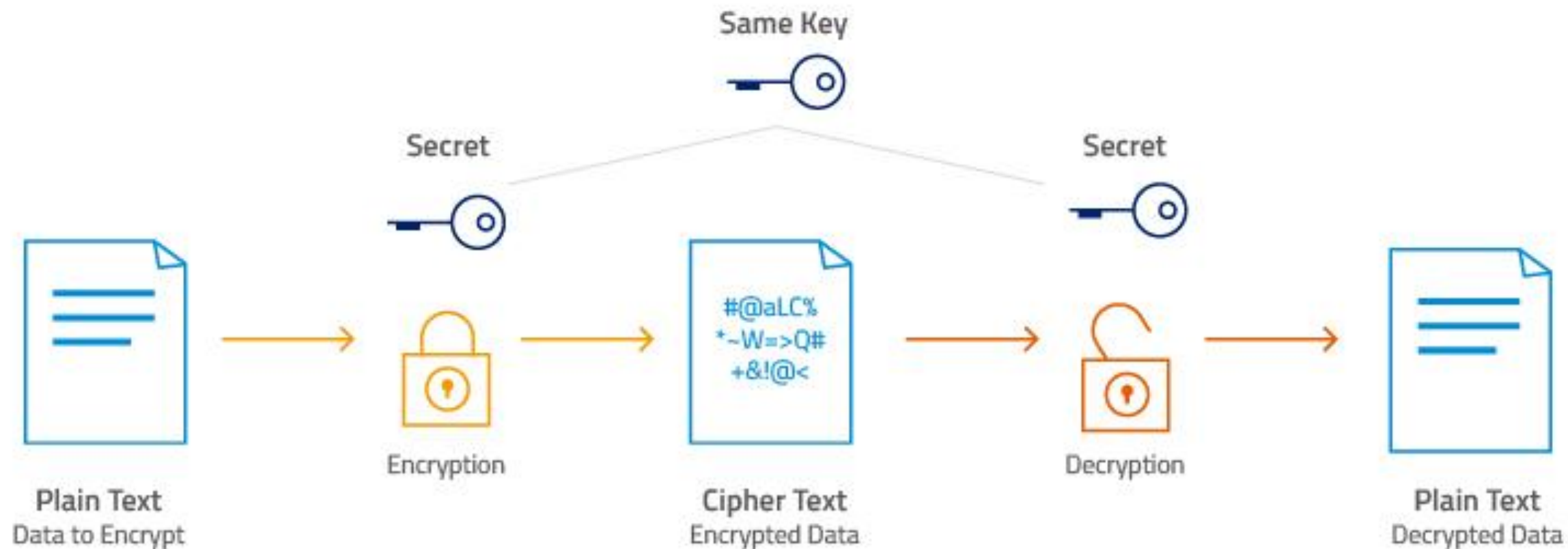
Wix uses Transport Layer Security and does meet PCI DSS compliance standards. We allow a minimum of TLS 1.2 so that out-of-date devices or browsers can access the website on your domain when they try to view it. At the same time, our sites allow the latest version of TLS for browsers which support it.

Visitors with older browsers may be able to visit your site, however, if their older device protocols don't meet PCI security standards, they may not be able to complete a transaction.



Security Measures

AES-256 File Encryption: All uploaded image content is encrypted at rest using industry-standard Advanced Encryption Standard with a 256-bit key.



Security Measures

SHA-256 Integrity Hashing: Cryptographic hashes verify file integrity, detecting any unauthorized alterations or corruption.

How Hashing Works



Gollum's Riddle
(Input)



Hash Function
(Hashing Algorithm)



Hash Value
(Output)

Security Measures

Audit Logging: Detailed logs capture timestamps, IP addresses, and transaction statuses for full compliance and accountability.

 pg-mmedcon

PostgreSQL 16.9

EOL : 9 November 2028 : OK

Running

Nodes 1

...

My Organization / finance25 / pg-mmedcon / Logs

Enable logs integration

Logs

[13-1] pid=50041,user=postgres,db=defaultdb,app=system-stats,client=[local] LOG: disconnection: session time: 0:00:00.065 user=postgres database=defaultdb host=[local]

[pg-mmedcon-5] 2025-07-20T18:14:51Z [postgresql-16]

[11-1] pid=50046,user=[unknown],db=[unknown],app=[unknown],client=[local] LOG: connection received: host=[local]

[pg-mmedcon-5] 2025-07-20T18:14:51Z [postgresql-16]

[12-1] pid=50046,user=postgres,db=finance,app=[unknown],client=[local] LOG: connection authorized: user=postgres database=finance

[pg-mmedcon-5] 2025-07-20T18:14:51Z [postgresql-16]

[13-1] pid=50046,user=postgres,db=finance,app=system-stats,client=[local] LOG: disconnection: session time: 0:00:00.011 user=postgres database=finance host=[local]

[pg-mmedcon-5] 2025-07-20T18:14:51Z [postgresql-16]

[11-1] pid=50047,user=[unknown],db=[unknown],app=[unknown],client=[local] LOG: connection received: host=[local]

[pg-mmedcon-5] 2025-07-20T18:14:51Z [postgresql-16]

[12-1] pid=50047,user=postgres,db=mMedconv2,app=[unknown],client=[local] LOG: connection authorized: user=postgres database=mMedconv2

[pg-mmedcon-5] 2025-07-20T18:14:51Z [postgresql-16]

[13-1] pid=50047,user=postgres,db=mMedconv2,app=system-stats,client=[local] LOG: disconnection: session time: 0:00:00.012 user=postgres database=mMedconv2 host=[local]

[pg-mmedcon-5] 2025-07-20T18:14:51Z [postgresql-16]

[11-1] pid=50048,user=[unknown],db=[unknown],app=[unknown],client=[local] LOG: connection received: host=[local]

[pg-mmedcon-5] 2025-07-20T18:14:51Z [postgresql-16]

[12-1] pid=50048,user=postgres,db=mmedcon,app=[unknown],client=[local] LOG: connection authorized: user=postgres database=mmedcon

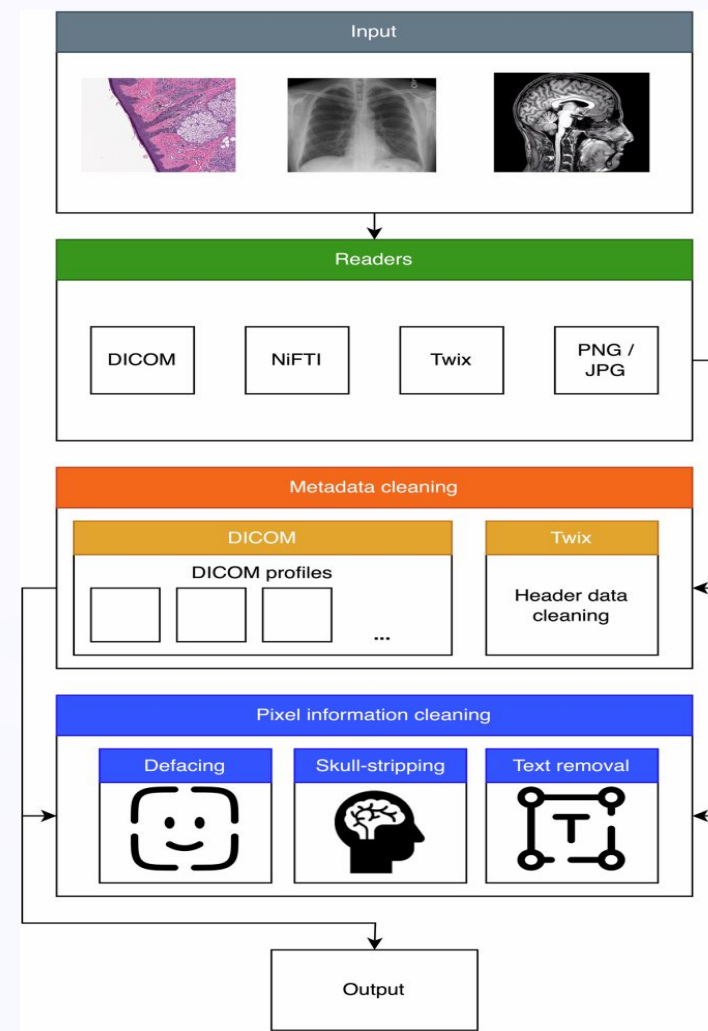
[pg-mmedcon-5] 2025-07-20T18:14:51Z [postgresql-16]

[13-1] pid=50048,user=postgres,db=mmedcon,app=system-stats,client=[local] LOG: disconnection: session time: 0:00:00.010 user=postgres database=mmedcon host=[local]

Go to most recent message

Security Measures

DICOM De-identification: Automated processes remove all Protected Health Information (PHI) from DICOM headers, safeguarding patient privacy.



Microservice Codebase Architecture

The FastAPI microservice is modular:

main.py: Contains the primary FastAPI application logic, defining API endpoints for file uploads, processing, and result retrieval.

dicom_utils.py: Encapsulates all DICOM-specific functionalities, including conversion from various image formats, anonymization of patient data, and file encryption routines.

database.py: Manages interactions with the PostgreSQL database, providing functions for inserting and retrieving data from the 'uploads', 'dicom_metadata', 'ml_results', and 'audit_log' tables.

Dockerfile: Specifies the environment and dependencies for containerizing the FastAPI service, ensuring consistent deployment across different platforms.

API Response: Designed to return machine learning analysis results immediately within the API response, enabling real-time feedback to users.

Database Schema Overview

The PostgreSQL database is structured to efficiently manage all pipeline data.

Table Name	Purpose and Key Fields
uploads	Tracks all uploaded files, including unique upload IDs, original filenames, and storage paths. Key fields: upload_id (PK) , file_name, stored_path, upload_timestamp.
dicom_metadata	Stores extracted DICOM headers and processing details post-conversion and anonymization. Key fields: dicom_id (PK) , upload_id (FK), patient_age, study_description, anonymization_status.
ml_results	Records the outcomes of machine learning analyses, including diagnosis and confidence scores. Key fields: result_id (PK) , dicom_id (FK), diagnosis_label, confidence_score, ml_model_version.
audit_log	Maintains a comprehensive log of all system actions for compliance and security auditing. Key fields: log_id (PK) , event_timestamp, user_ip, action_type, status, details.

Example Upload Response

Upon successful upload and processing, the API returns a structured JSON response.

```
{  "upload_id": "ab12c3d4e5f6g7h8i9j0k1l2m3n4o5p6",  "message": "Upload and processing complete.",  
  "diagnosis": "Tumor Detected",  "confidence": 0.89}
```

This response provides immediate confirmation of the upload, details the processing status, and includes the preliminary machine learning diagnosis with its associated confidence score, enabling prompt user feedback and subsequent actions.

Deployment Strategy

Our pipeline is built for flexible and scalable deployment.



Containerization

The entire service will be containerized using Docker, ensuring portability and consistent operation across diverse environments.



Reverse Proxy Integration

Can be deployed behind a reverse proxy like NGINX for load balancing and enhanced security.



Cloud Agnostic

Future deployments can leverage secure cloud platforms such as AWS or Google Cloud, integrating with services like S3 for storage and advanced load balancing.

Test

Test image

```

# Import necessary modules
import os
import psycopg2
import json
import logging
import requests

# Load environment variables from .env file
def load_env():
    env_file = '.env'
    if os.path.exists(env_file):
        with open(env_file, 'r') as f:
            for line in f:
                line = line.strip()
                if line and not line.startswith('#'):
                    key, value = line.split('=')
                    os.environ[key] = value

load_env()

# Database connection
def get_db_connection():
    conn = psycopg2.connect(
        dbname=os.getenv("POSTGRES_DB", "mmedconv2"),
        user=os.getenv("POSTGRES_USER", "dummy_user"),
        password=os.getenv("POSTGRES_PASSWORD", "dummy_password"),
        host=os.getenv("POSTGRES_HOST", "dummy_host"),
        port=os.getenv("POSTGRES_PORT", "5432")
    )
    return conn

# Main function
def main():
    conn = get_db_connection()
    cursor = conn.cursor()

    # Upload record
    def upload_record(upload_id, filename, ext, upload_time, ip, storage_path, sha256_hash):
        cursor.execute("""
            INSERT INTO uploads (id, original_filename, file_type, upload_time, uploader_ip, storage_path, sha256_hash, encrypted, status)
            VALUES (%s, %s, %s, %s, %s, %s, %s, TRUE, 'processed')
        """)
        conn.commit()

    # Upload metadata
    def upload_metadata(upload_id, dicom_converted, anonymized, removed_tags, processed_at):
        cursor.execute("""
            INSERT INTO dicom_metadata (upload_id, dicom_converted, anonymized, removed_tags, processed_at)
            VALUES (%s, %s, %s, %s, %s)
        """)
        conn.commit()

    # Upload result
    def upload_result(upload_id, model_version, diagnosis, confidence, analyzed_at):
        cursor.execute("""
            INSERT INTO ml_results (upload_id, model_version, diagnosis, confidence_score, analyzed_at)
            VALUES (%s, %s, %s, %s, %s)
        """)
        conn.commit()

    # Log action
    def log_action(upload_id, action, timestamp, ip, status, details):
        logging.info(f"Action: {action} at {timestamp} from {ip}. Status: {status}. Details: {details}")

    # Test upload
    upload_id = "test_upload"
    filename = "brain.png"
    ext = ".png"
    upload_time = datetime.now()
    ip = "127.0.0.1"
    storage_path = "C:/Users/USER/Desktop/mmedconv2-figs/brain.png"
    sha256_hash = "9a16103b-68c3-42cf-b66c-37f823b4b87c"

    upload_record(upload_id, filename, ext, upload_time, ip, storage_path, sha256_hash)
    upload_metadata(upload_id, dicom_converted, anonymized, removed_tags, processed_at)
    upload_result(upload_id, model_version, diagnosis, confidence, analyzed_at)
    log_action(upload_id, "upload", timestamp, ip, status, details)

    # Test download
    def download_record(upload_id):
        cursor.execute("""
            SELECT * FROM uploads WHERE id = %s
        """)
        record = cursor.fetchone()
        return record

    record = download_record(upload_id)
    if record:
        print(f"Record found: {record}")
    else:
        print("Record not found")

    # Test delete
    def delete_record(upload_id):
        cursor.execute("""
            DELETE FROM uploads WHERE id = %s
        """)
        conn.commit()

    delete_record(upload_id)

    # Test update
    def update_record(upload_id, new_filename, new_ext, new_upload_time, new_ip, new_storage_path, new_sha256_hash):
        cursor.execute("""
            UPDATE uploads SET original_filename = %s, file_type = %s, upload_time = %s, uploader_ip = %s, storage_path = %s, sha256_hash = %s WHERE id = %s
        """)
        conn.commit()

    update_record(upload_id, new_filename, new_ext, new_upload_time, new_ip, new_storage_path, new_sha256_hash)

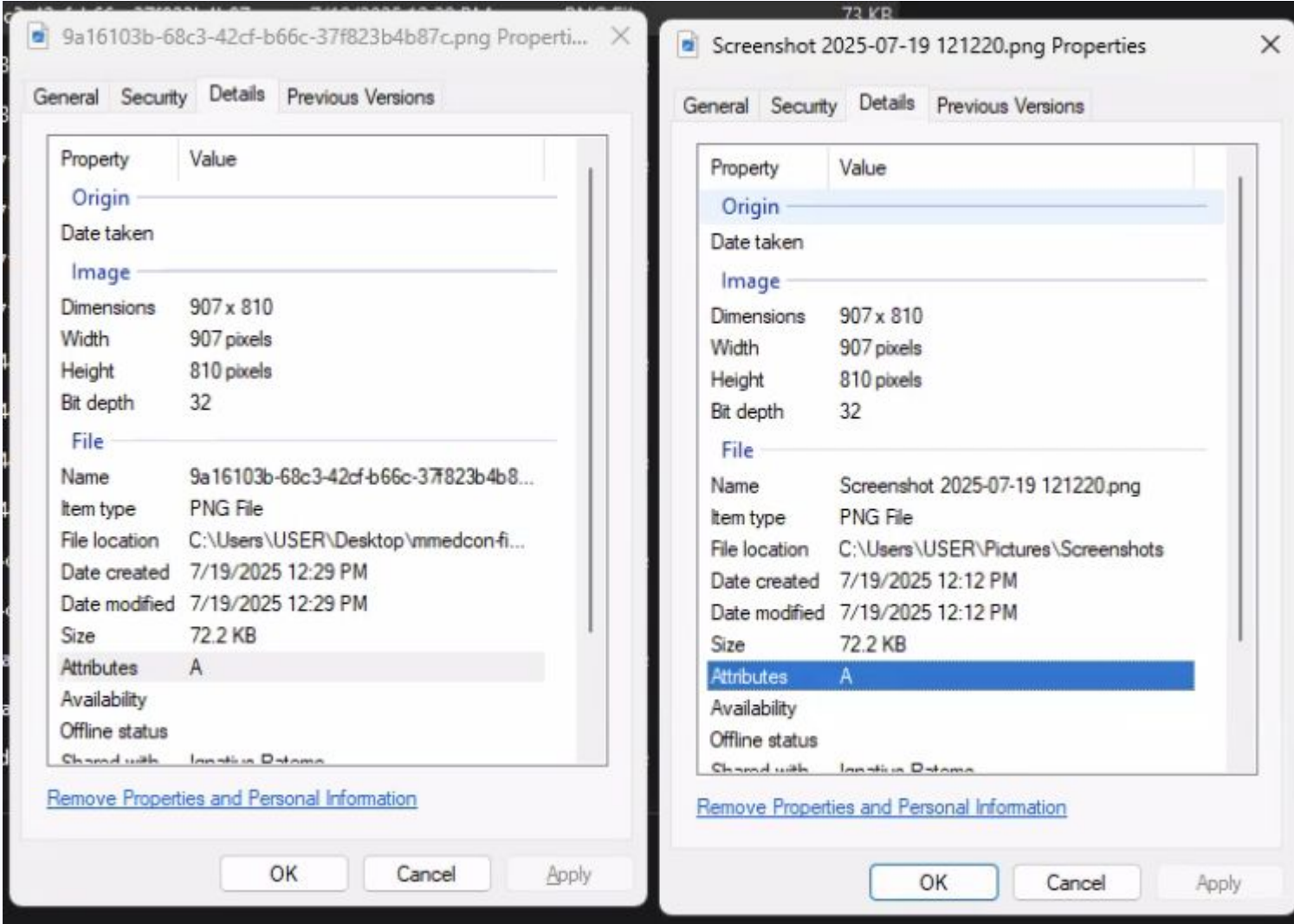
    # Test query
    cursor.execute("""
        SELECT * FROM uploads
    """)
    records = cursor.fetchall()
    for record in records:
        print(record)

    cursor.close()
    conn.close()

if __name__ == "__main__":
    main()

```

Check the resolution before and after upload



Future Enhancements

Our roadmap includes key improvements to evolve the pipeline.

Real ML Model Integration: Replace the mock ML model with a production-ready, validated diagnostic model for accurate analysis.

Real-time Status Updates: Implement mechanisms for users to receive real-time updates on their image processing status via websockets or push notifications.

Integrated Image Viewer: Develop an interactive image viewer within the results dashboard for direct examination of processed images.

HIPAA-Compliant File Hosting: Transition to certified HIPAA-compliant cloud storage solutions for robust data governance.

FHIR/EHR Integration: Enable seamless integration with Fast Healthcare Interoperability Resources (FHIR) or Electronic Health Record (EHR) systems for comprehensive patient data management.

Check the resolution before and after upload: Enable seamless integration with Fast Healthcare Interoperability Resources (FHIR) or Electronic Health Record (EHR) systems for comprehensive patient data management.

Thank you